프로젝트

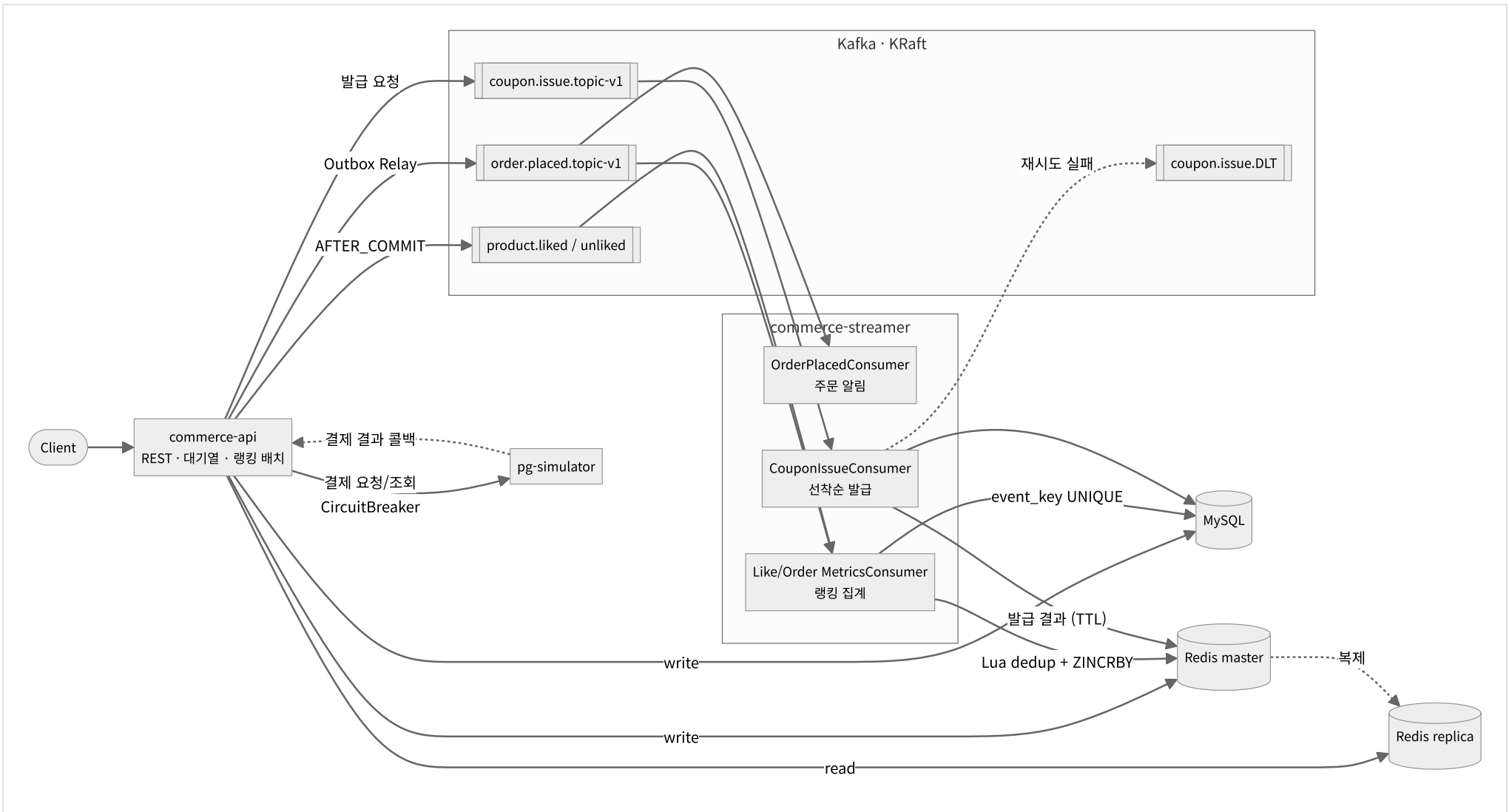
Loopers Commerce Backend

Loopers 백엔드 부트캠프 10주　Java 21 · Spring Boot 3.4 · JPA/QueryDSL · MySQL 8.0 · Redis · Kafka · Resilience4j · k6

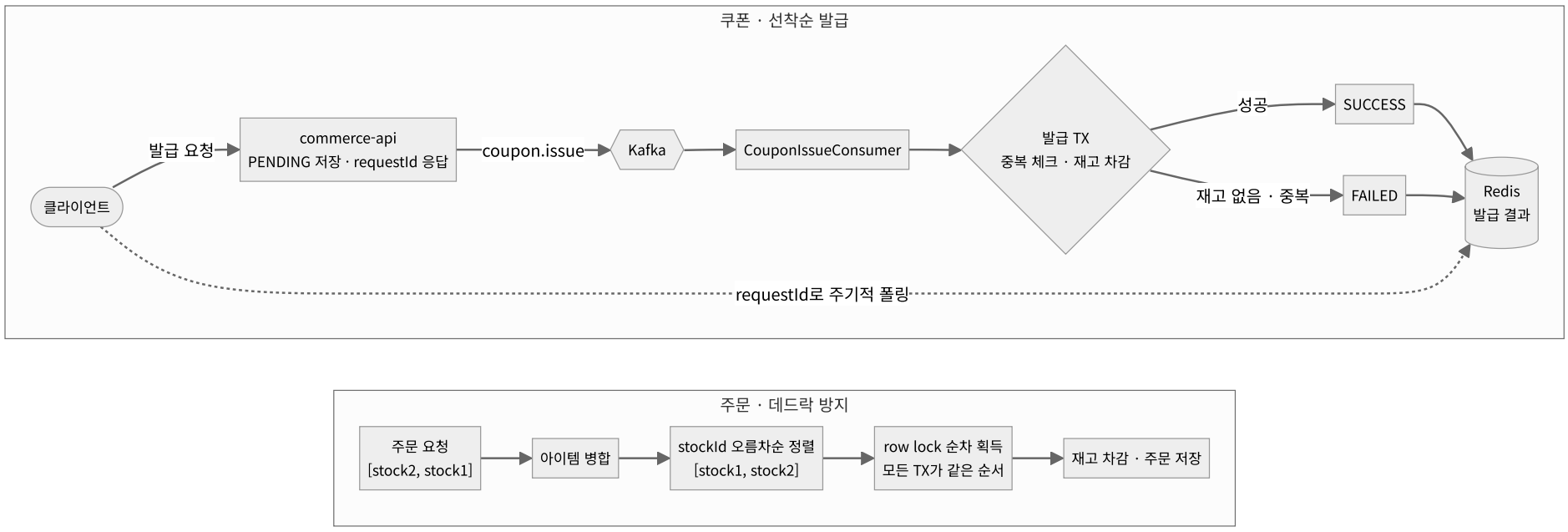
github.com/sun8183/loop-pack-be-l2-vol4-java

이커머스 백엔드 멀티모듈 프로젝트. 동시성 제어, 대규모 트래픽 대응, 이벤트 기반 데이터 정합성까지 단계적으로 확장하며, 문제 재현과 부하테스트 실측으로 운영값을 산정하는 데 중점을 둔 프로젝트

시스템 구조



1. 선착순 쿠폰 동시성 제어 & 멀티 아이템 주문 데드락 해결



배경

- 선착순 쿠폰 오픈 직후 요청이 몰려, 요청마다 DB 재고를 차감하면 락 경합이 DB 부하로 직결되는 문제 존재
- 같은 유저가 N번의 쿠폰 발급 요청으로 여러 건의 쿠폰 중복 발급 위험 존재
- 다중 상품 주문 시 요청 순서대로 row lock을 잡아, 역순 주문끼리 교착 상태 및 간헐적 500 발생

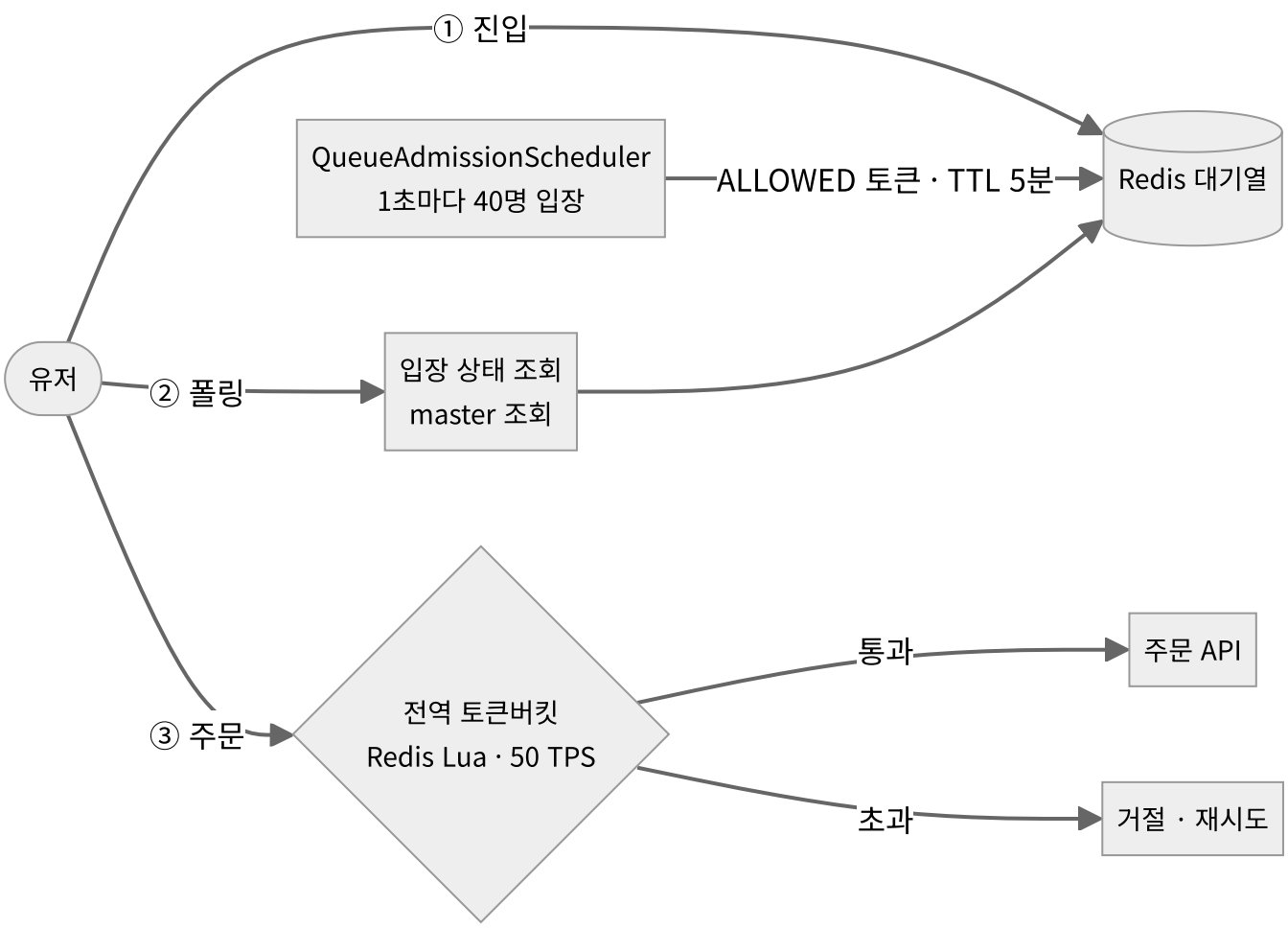
구현

- 발급 요청을 Kafka 단일 파티션으로 받아 컨슈머가 한 건씩 재고 차감
- 클라이언트는 requestId로 발급 결과를 주기적으로 폴링
- 컨슈머 처리 시 DB 유니크 제약(user_id, coupon_id)으로 쿠폰 중복 발급 방지
- 주문 항목을 stockId 오름차순으로 사전 정렬해 모든 트랜잭션의 lock 획득 순서 통일

결과

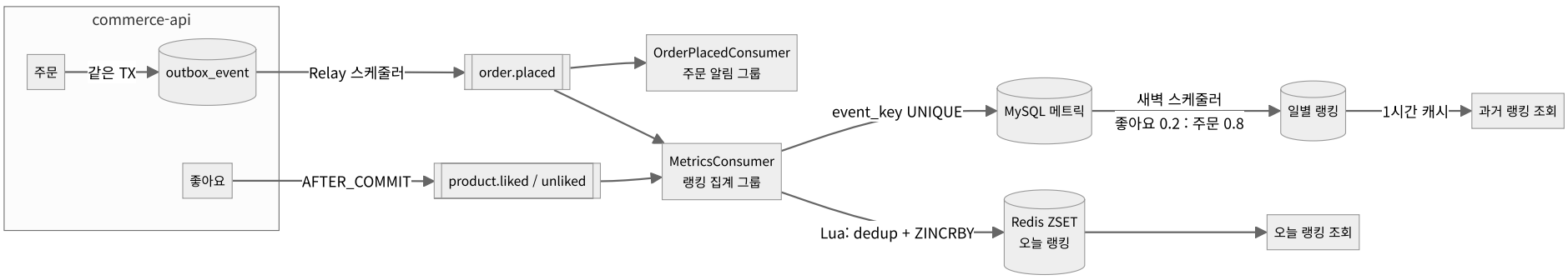
- 선착순 쿠폰 오픈 직후 요청이 몰리는 현상 완화
- 재고 100개 쿠폰에 200명 동시 발급 요청 시 정확히 100명만 성공
- 역순 동시 주문에서 발생하던 데드락 해소

2. 주문 대기열 & 전역 Rate Limiter (부하테스트 기반 용량 산정)



배경	• 블랙프라이데이처럼 주문 트래픽이 순간 폭주하는 가상 시나리오를 설정하고, 서버·DB가 감당 가능한 수준으로 유입을 제어할 구조 필요 • 대기열은 입장 속도만 제어해, 입장 후 주문 몰림은 막지 못함
구현	• 대기열은 REDIS로 토큰 검증·폴링 API 구현, 스케줄러 batch-size로 입장 속도 조절 • k6 단계별 고정 부하 테스트(RATE 20~80)로 서비스 처리 한계 측정 • 상품 주문 요청 시 Redis Lua 전역 토큰버킷(capacity/refill=50)으로 2단 방어
결과	• RATE 50까지 p95 100~140ms로 안정, RATE 60부터 13.9초로 급증해 안전 상한 50 TPS 확인 • 실패율은 전 구간 0.5%대로 일정해 지연시간을 용량 경계 지표로 판단, 10 TPS 마진을 두고 batch-size=40 확정하여 2단 제어의 기준값 산정

3. 이벤트 기반 실시간 랭킹 시스템



배경

- 주문·좋아요 트랜잭션에서 랭킹 집계까지 처리하면 응답 지연과 도메인 간 결합 증가
- 이벤트마다 정합성 요구가 다름: 주문은 알림·랭킹이 함께 써서 유실 불가, 좋아요는 랭킹 전용

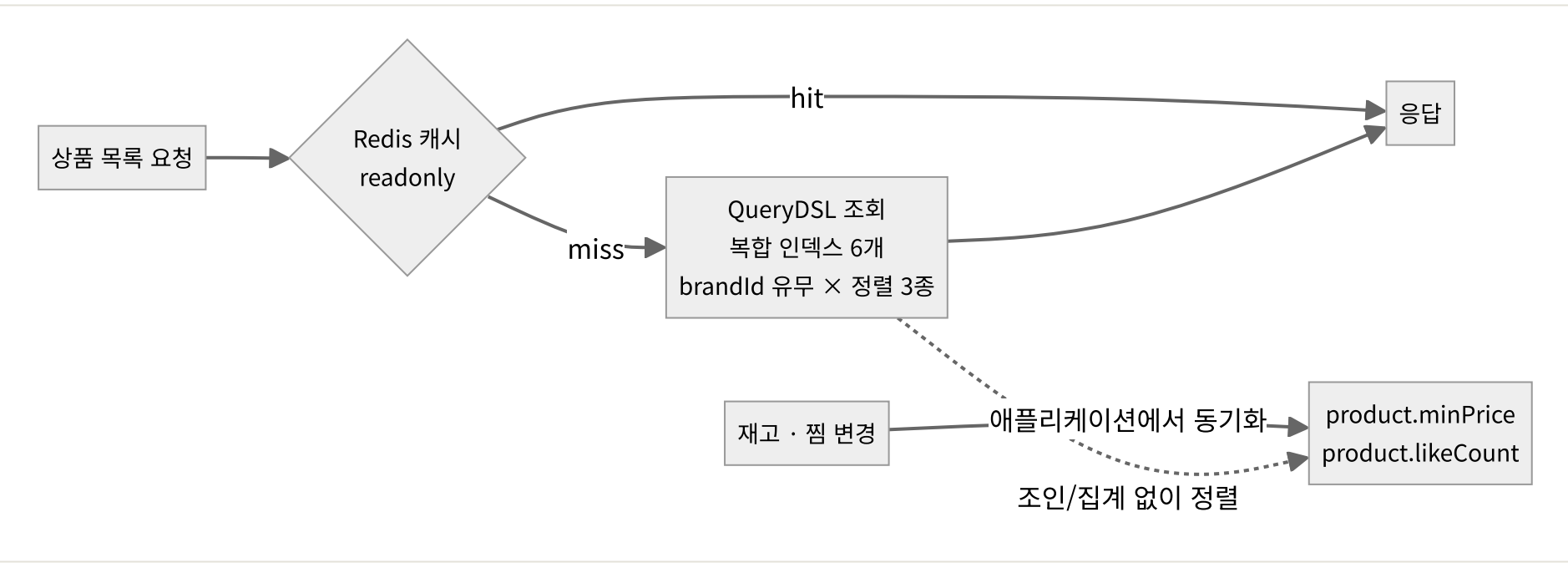
구현

- Kafka로 이벤트 발행, commerce-streamer가 소비해 랭킹 반영 (주문 생성 이벤트는 알림·랭킹 컨슈머 그룹이 각각 구독)
- 주문은 Outbox 패턴, 좋아요는 AFTER_COMMIT 시점 발행
- 재처리(배치 실패·재시작)·재발행으로 같은 이벤트가 다시 와도 중복 반영 차단: Redis는 Lua로 이력 저장과 점수 증가를 한 번에 실행, DB는 event_key UNIQUE로 구현
- 랭킹 점수는 주문 0.8·좋아요 0.2 가중치로 계산: 오늘 랭킹은 Redis ZSET에 실시간 누적, 과거 랭킹은 새벽 스케줄러가 DB에 쌓인 횟수로 재계산 후 1시간 캐싱

결과

- 주문은 발행 유실 없이, 재발행·중복 소비에도 1회만 반영되도록 설계
- 좋아요는 드문 발행 유실을 허용하는 대신 outbox 없이 단순한 구조 유지

4. 상품 조회 성능 최적화



배경

- 상품 10만·좋아요 300만 건 환경에서 목록 조회가 좋아요 수를 매번 집계해, 좋아요순 정렬은 JOIN·GROUP BY로 1.52초 소요
- 복합 인덱스 4개를 추가했지만 좋아요순은 1.57초로 개선 없음 → 병목이 정렬이 아닌 집계에 있음을 확인

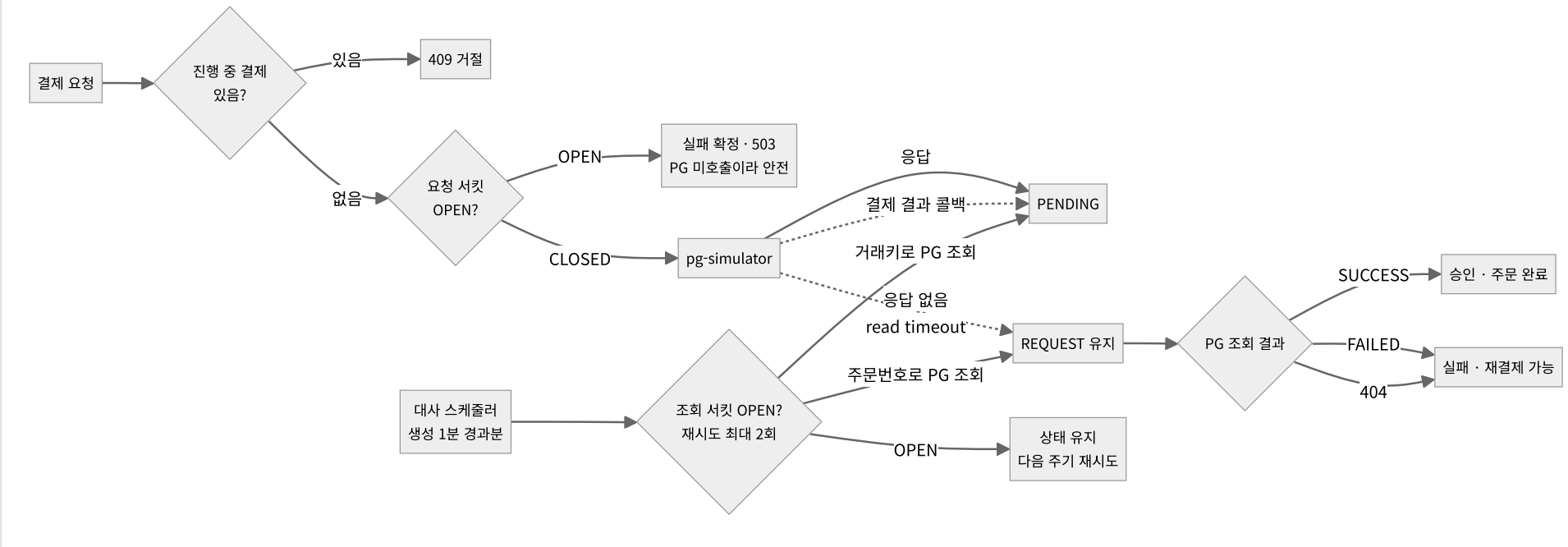
구현

- k6로 정렬 3종 × 브랜드 필터 유무 6개 조합을 단계별로 측정하며 개선 효과 검증
- 좋아요 테이블 인덱스가 user_id 선두라 상품 기준 집계에 쓰이지 않던 문제를 product_id 선두 인덱스로 해결
- likeCount를 상품 테이블에 반정규화하고 좋아요 추가·취소 시 원자적 연산으로 갱신, brandId 유무 × 정렬 3종 복합 인덱스 6개 구성
- 요청이 몰리는 첫 페이지만 정렬별로 Redis 캐싱

결과

- 좋아요순 조회 1.52초 → 117ms (브랜드 필터 시 1.01초 → 46ms), 최신순 515ms → 119ms
- 좋아요 쓰기는 119ms → 208ms로 느려졌지만, 조회 비중이 높아 반정규화를 선택

5. PG 결제 연동: 결과 불명 결제의 상태 대사



배경

- 결제 요청이 타임아웃되면 실패로 처리했으나, PG는 이미 승인했을 수 있어 재결제 시 이중 결제 위험 존재
- 결제 진행 중에도 같은 주문으로 결제 요청이 다시 나갈 수 있었음
- PG 콜백은 실패해도 재시도하지 않아, 콜백만으로는 결제 결과가 확정되지 않는 경우 존재

구현

- PG가 멱등키를 제공하지 않아 결제 요청은 재시도에서 제외하고, 응답을 받지 못한 결제는 실패가 아닌 결과 불명(REQUEST)으로 유지
- REQUEST·PENDING 결제를 복구하는 대사 스케줄러 설계: 거래키가 없는 REQUEST는 주문번호로, PENDING은 거래키로 PG를 조회해 상태 확정 (생성 1분 경과분 대상)
- 주문번호로 조회했을 때 PG에 해당 주문의 거래가 없으면(404 응답) 요청이 PG에 도달하지 않은 것으로 판단해 실패 처리
- 주문당 진행 중 결제를 1건으로 제한해 결과 확정 전 재결제 차단
- 결제 요청 서킷: 최근 10건 중 5건 이상 호출 시 실패율 50% 이상이면 30초간 OPEN, 이후 3건 시험 호출로 복구 판단
- 결제 조회 서킷: 최근 15건 중 6건 이상 호출 시 실패율 50% 이상이면 10초간 OPEN, 이후 2건 시험 호출로 복구 판단 (재시도 1회도 실패 1건으로 집계)
- 4xx(HttpClientErrorException)와 도메인 예외(CoreException)는 PG 장애가 아니므로 실패율 집계에서 제외

결과

- 타임아웃·콜백 유실이 발생해도 PG 상태와 일치하도록 결제를 확정
- 서킷으로 PG 장애를 격리해 결제 API 전체로 번지지 않도록 설계: OPEN 시 최대 5초의 타임아웃 대기 없이 즉시 503 응답